

DYAD

Relational intelligence — the first software that operates on the real machinery of the relationships you can't afford to lose.

Development Design Document

BUILD WINDOW · 12 HOURS · STACK: GSTACK / GBRAIN / THE HOG / JO · TARGET: TOP 1, YC INTERVIEW

EXECUTIVE SUMMARY

The one-page version

Dyad is a mobile-first relational intelligence platform. It ingests the user's real communication history with the specific humans in their life — partner, co-founder, parent, close friend — and produces a continuously updated, calibrated, citation-backed model of how that relationship is actually functioning. The system surfaces what is silently going wrong, in time for the user to do something about it.

The category does not exist in the Y Combinator portfolio of 669 current companies. Personal-AI products know the user (jo, Pickle). Persona simulators model fictional people (Auxos, Expected Parrot, Artificial Societies). Clinical AI sits on the practitioner side (Klarify). Nothing operates on the dynamics between real, named humans who actually have to navigate one another. That gap is the wedge.

The product is built on validated relationship science — Gottman on rupture and repair, Reis & Shaver on perceived partner responsiveness, Sue Johnson's EFT framework on primary versus secondary emotion, Mikulincer & Shaver on attachment under threat, Walton & Wilson on wise interventions, Pennebaker on function-word linguistics. Each detector in the system is grounded in a specific literature and surfaces calibrated confidence. This is not vibes-based AI; it is operationalized cognitive science.

Six demo-critical features ship in the twelve-hour build window: the Map (longitudinal emotional topography), the Emotion Atlas (per-message labeled emotion with intensity and secondary-emotion inference), Bid Response Asymmetry (Gottman's most empirically grounded metric), Primary/Secondary Emotion Separation (the real fight underneath the stated fight), Predictive Divergence (the literal numeric distance between intent and reception), and the Phantom Third Party (transferential pattern detection — the parent's voice in the partner's criticism). A seventh structural feature, the ethical refusal layer, draws a visible clinical line on stage.

The build is parallelized across a small team plus frontier coding agents working through GStack. The mobile experience is the primary surface; iMessage is the primary data source, with Discord, WhatsApp, Slack, and email scaffolded for the post-hackathon roadmap. The demo runs on a real volunteer relationship first and a pre-loaded public-figure corpus second — variance reduction is treated as a first-class engineering problem, not an afterthought.

The pitch ends with a vertical-expansion path that is legible to a venture audience: consumer wedge → co-founder relationship monitoring (the #1 cited cause of early-stage startup failure) → high-stakes negotiation → sales call intelligence → mediation → therapy augmentation. The moat is longitudinal dyad data nobody else can replicate. The defensibility compounds with every conversation observed.

Target outcome: top-three hackathon finish, with the YC interview as the real prize. Probability of the interview, on this plan, executed cleanly, is meaningfully higher than the probability of winning the room — because Garry's filter is "is this interesting and is this problem real and is this team thoughtful," and on all three this plan is strong.

PART I

Vision & Strategy

1.1 The problem, stated honestly

Most adult suffering happens in dyads. Not the dramatic kind that makes the news — the slow, ordinary kind that constitutes the central tragedies of adult life. Estranged parents. Co-founder breakups that destroy companies. Marriages that ended without warning, where one party can name the moment the other already checked out, and the other party cannot. Friendships that ghosted. Adult children who do not call. Siblings who stopped speaking after a small thing that became a permanent thing because nobody named it.

These failures share a structure. The dynamics that govern the relationship are invisible to the people inside it. Each party has a model of the other that is calibrated against the wrong evidence. The patterns repeat. The repairs never happen. The window closes.

The clinical literature on this is unusually well-developed. John Gottman has spent four decades quantifying which interaction patterns predict divorce, with accuracy that exceeds 90% on short conversation samples. Reis and Shaver established perceived partner responsiveness as the single strongest predictor of relational outcomes. Sue Johnson's emotion-focused therapy framework distinguishes between primary emotions (the vulnerable ones underneath: hurt, fear, shame) and secondary emotions (the protective ones expressed: anger, contempt, withdrawal) and explains most chronic conflict as primary-emotion misreading. Edward Tronick's work on the still-face paradigm demonstrated that secure attachment is built not by avoiding rupture but by repairing it — and that the rate of successful repair, not the rate of rupture, predicts long-term outcomes.

None of this is in software. All of it could be.

1.2 The category gap

A scan of the 669 current Y Combinator portfolio companies, focused on the cognitive, social, and emotional AI segment, reveals four consistent shapes:

- **Personal-AI products that know the user.** jo, Pickle, and adjacent products model the user's own context, preferences, and history. The unit of analysis is one person.
- **Persona simulators that model fictional people.** Auxos, Expected Parrot, and Artificial Societies generate synthetic audiences for research or marketing. The humans simulated are not real and the outputs are not verifiable against ground truth.
- **Clinical AI that sits on the practitioner side.** Klarify augments therapists; Patientdesk and similar serve providers. The patient or partner is the object of care, not the user of the tool.
- **Wellness companions that coach the individual.** Sunflower, Nori, Prana, Juno, and the broader category coach the user toward a goal. The relationship to other humans is incidental.

Across all 669, the dynamics between two real, named humans who actually have to navigate each other are not the unit of analysis for any shipped product. The space between people — which is where most of human life actually happens — is empty.

This is the wedge. It is not adjacent to a saturated market; it is structurally vacant. Building it well requires three things that most teams cannot bring simultaneously: real engineering on live data, a working command of the relationship-science literature, and the ethical discipline to refuse the easier manipulation-adjacent product that

this stack could otherwise produce.

1.3 Why now, why this stack

The four building blocks provided by the hackathon are unusually well-suited to this specific product, in ways that are not coincidental. **jo** provides the private-context surface — access to the user's real messages, calendar, and personal artifacts — without which a relational model is decorative. **GBrain** provides the persistent, entity-resolved memory layer that turns a series of one-shot inferences into a longitudinal model of a specific dyad. **The Hog** provides the external-context layer that contextualizes the other party beyond what they say to the user — what they've been reading, posting, engaging with, signaling. **GStack** provides the orchestration spine that makes a system of this architectural complexity tractable in twelve hours.

Used together, the four blocks compose into something none of them is alone. jo plus GBrain is a personal assistant with memory; useful, not novel. jo plus GBrain plus The Hog is the first infrastructure for modeling a real dyad over time with both parties' full context represented — one through their actual messages, the other through what their public footprint reveals about the world shaping them. That composition is the product.

1.4 Why this team, why mobile, why now

The team is composed of engineers who already work with frontier coding agents as a baseline practice, which changes the per-engineer production rate by roughly an order of magnitude on focused, well-specified work. This makes a twelve-hour build of a system with this many moving parts realistic. It also means the architecture should be designed to be agent-parallelizable, with clean module boundaries and explicit specifications — which is how this DDD is structured.

The product is mobile-first because the data is mobile-first. People do not text on desktops. iMessage is the primary substrate for the kind of intimate, longitudinal, high-stakes communication this product operates on. An iOS-native experience that surfaces relational intelligence in the place where the conversations actually happen is the right shape. The hackathon demo will be shown on a phone, in a phone frame — not because that is how engineers want to demo software, but because that is how the user will actually experience the product.

Now is the moment because three things became simultaneously true in the last twelve months: foundation models can perform linguistic analysis on intimate communication with calibrated confidence; persistent memory infrastructure (GBrain) has matured to the point where longitudinal modeling is cheap; and the consumer permission structure for personal AI (jo and predecessors) has normalized the act of giving an agent access to private channels. None of these were true two years ago. All three are required.

PART II

Product Specification

2.1 The product, in one paragraph

Dyad runs as a passive, asymmetric, longitudinal intelligence layer on top of the user's real communication channels. It ingests messages via jo, persists state in GBrain, contextualizes the other party via The Hog, and is orchestrated by GStack. It never sends messages, never impersonates, never auto-replies. It does exactly one thing: it makes the invisible machinery of the user's relationships visible to the user, in time to act on it. The product's deepest output is never "here is what they are doing wrong." It is always "here is what you are doing that you cannot see, and here is what the data says is true between the two of you."

2.2 The six demo-critical features

Out of the twenty-one upgrade ideas evaluated during planning, six make the demo. The selection criteria were: (a) does this feature fire reliably across many relationships; (b) is it visually legible in under fifteen seconds; (c) does it produce an output that a judge can verify against their own knowledge; (d) is it grounded in citeable empirical work.

Feature 1 — The Map

What it is. The user's relationship rendered as a single scrollable visualization: emotional temperature over time, rupture-and-repair events marked as discrete points, topic-drift regions shaded, bid-and-response cadence shown as rhythm. The vertical axis is emotional valence; the horizontal axis is time; color encodes the dominant emotion from the Emotion Atlas; the rendered surface animates through time on first load, with months of relational dynamics compressing into roughly eight seconds.

Why it matters. People have never seen their relationships from above. The first time someone watches a fourteen-month arc render as a literal shape, something shifts that does not unshift. This is the demo's primary visual hook and the answer to the legibility problem — it lands before any explanation, in the first ten seconds, while the judge is still building context.

Citation grounding. The longitudinal-view approach is informed by Boals, Banks, and Hayslip's work on narrative coherence in autobiographical memory: people who can construct coherent narratives about difficult relational events show better outcomes than people who cannot. The Map exists to support that construction visually.

Feature 2 — The Emotion Atlas

What it is. Every message in the corpus carries a tag for primary emotion (drawn from Plutchik's eight: joy, trust, fear, surprise, sadness, disgust, anger, anticipation), intensity (low / medium / high), and an inferred secondary emotion where the surface emotion is identified as a protective layer over a vulnerable one. The atlas surfaces in three places: as per-message color tags in the conversation view, as a moving heat band along the Map, and as an aggregate "emotional signature" card showing the dominant emotional textures of the relationship over rolling windows.

Why it matters. Emotion labeling is the single feature that makes the entire product immediately legible to anyone in the room regardless of their familiarity with relationship science. A judge does not need to know what perceived partner responsiveness is to understand that the last three weeks of a relationship are shaded

predominantly in muted blue and ochre. Visceral output beats intellectual output in a competitive demo environment. This feature is the bridge.

Citation grounding. Plutchik's wheel for the primary categories; Ekman's basic emotions literature for the intensity gradient; Sue Johnson's EFT primary-versus-secondary framework for the under-the-surface tag; Higgins's self-discrepancy theory for the dejection-versus-agitation family-of-emotion classification used to inform the secondary inference.

Implementation notes. A two-stage classifier — a fast, cheap function-word and lexicon pass (using NRC Emotion Lexicon and an open EmoLex-equivalent) to produce a coarse tag, followed by an LLM verification pass on uncertain or high-stakes messages to refine the tag and infer the secondary emotion. The fast layer keeps cost and latency manageable across a full corpus; the LLM layer ensures the demo-critical messages are accurately tagged. Every tag carries a confidence score, surfaced as opacity in the visualization.

Feature 3 — Bid Response Asymmetry

What it is. A pair of numbers: the rate at which the user responds to the partner's bids for connection, and the rate at which the partner responds to the user's. Both are shown against Gottman's empirical thresholds — 86% in stable long-term relationships, ~33% in failing ones — with the specific bids and the specific responses (or non-responses) linkable for verification.

Why it matters. This is the most empirically grounded single number in relationship science. It is also nearly always shocking when first surfaced, because the asymmetry is almost always significant and one party is almost always unaware. In demo testing, bid asymmetry is the most consistently reliable “the volunteer writes something down” moment in the first ninety seconds.

Citation grounding. Gottman's “The Relationship Cure” on bidding; Gottman and Levenson's longitudinal studies on predictive validity; subsequent replication work in the Journal of Family Psychology.

Implementation notes. A bid is a small attempt to engage: an observation, a question, a share, a request for attention. A response is any acknowledgment beyond perfunctory minimum (an “ok” does not count as a response to a bid). The classifier uses a fine-tuned prompt over a held-out set of annotated bid/response pairs from the relationship-science literature's public corpora plus our own manually labeled examples.

Feature 4 — Primary / Secondary Emotion Separation

What it is. For any flagged conflict in the corpus, the system separates the stated content (the surface argument) from the inferred underlying emotion (the vulnerable feeling being protected against). The output names both, cites the specific linguistic markers that support the inference, and proposes a reframe that addresses the primary rather than the secondary.

Why it matters. This is the feature that produces audible reaction in demo testing. When the system correctly identifies that “the fight on Tuesday was about the calendar; what they were actually upset about was not feeling prioritized,” and the volunteer recognizes it as true, the room understands that Dyad is operating at a level no other product operates at.

Citation grounding. Johnson's “Hold Me Tight” and the broader emotion-focused therapy literature; Higgins's self-discrepancy theory for the dejection-versus-agitation classification; Aronson's self-affirmation theory for the reframe-design principle.

Feature 5 — Predictive Divergence

What it is. For any draft message the user is about to send, the system runs two predictions: what the user intends to communicate, and what the partner-model says the partner will actually hear. The gap between the two is rendered visually — not as a paragraph, as a literal distance — with the specific phrases that diverge highlighted in both directions.

Why it matters. Nobody has ever shown a person the literal numeric distance between what they said and what was heard. This is the demo's "holy shit" moment, in the same way that seeing a recording of one's own voice for the first time is. The output is structurally novel.

Citation grounding. Reis and Shaver on perceived partner responsiveness as the gap between intended and received care; Vallacher and Wegner on action identification asymmetry; Murray and Holmes on risk regulation.

Feature 6 — The Phantom Third Party

What it is. When the user's reaction in a specific interaction is disproportionate to the trigger, and when the shape of the disproportion matches earlier-relationship templates inferred from longitudinal data, the system surfaces the transference pattern. "The reaction you had on Tuesday is structurally similar to three earlier ruptures in this relationship, and looking back across your message history with your father six months ago, the pattern matches."

Why it matters. This is the highest-risk and highest-reward detector in the system. When it fires correctly, it is the most jaw-dropping output the product can produce — recognizing transference is what makes good therapists feel like mind-readers. When it fires incorrectly, it is the most embarrassing. It is therefore designed with high confidence thresholds and explicit "low-confidence" suppression: if the system is not confident, it does not surface this output at all.

Citation grounding. Paul Wachtel's cyclical psychodynamics literature for the theoretical frame; classical transference work from psychodynamic clinical writing; Mikulincer and Shaver's attachment-style-under-threat behavioral markers for the pattern-matching layer.

2.3 The seventh feature: the ethical refusal

Beyond the six diagnostic features, a structural feature ships in the build that is not a detector but a refusal. When the system encounters patterns that cross into clinical territory — intimate-partner abuse markers, suicidality indicators, severe depression signatures — it declines to perform further analysis and surfaces appropriate referral resources. This is implemented as a real classifier built on the computational psychiatry literature (Coppersmith on suicidality detection from language; De Choudhury on depression markers in social media; validated abuse-pattern indicators from the intimate-partner-violence research literature).

It is shipped as a demonstrated feature in the demo, on a pre-seeded example that triggers the refusal live. Drawing this line visibly is itself the feature. Almost no other product in the room will do this. Judges who care about responsible AI (which, in this room, includes the most important judges) will specifically remember the team that demonstrated the discipline to stop on stage.

2.4 What is deliberately not in the MVP

Twenty-one features were considered in the planning phase. Fifteen are explicitly cut from the demo build, with reasoning, so the team is not tempted to add them at hour 11. They include:

- **Cross-relationship pattern transfer.** Showing that the user fights with their partner the same way they fought with their last partner the same way they fight with their co-founder. Requires data volunteers will not have on hand. Roadmap.
- **Live mid-conversation intervention.** Real-time suggestions during an active conversation. Ethical line: makes the user worse over time by outsourcing presence. Deliberately not built.
- **Repair attempt scoring at micro-grain.** Detecting and scoring individual repair bids within a conflict. Architecturally present in the bid-detection layer but not surfaced in the MVP UI.
- **Attachment-style classification UI.** The classifier runs internally to inform other detectors but is not user-facing in the MVP — the surface area is too easily misread as labeling.
- **Co-regulation timing recommendations.** “Wait 23 minutes before responding” outputs based on physiological recovery windows. In the data, not in the UI.
- **The four horsemen live classifier as a primary surface.** Runs internally and informs the Map; not the primary UI feature.
- **The asymmetric repair-labor index, as its own card.** Surfaces only as part of the Map narration, not as a separate view.
- **Eight additional features deferred to the post-hackathon roadmap.** Each named and dated in Part VIII.

The discipline of cutting is itself the feature. Teams that ship twenty mediocre features lose to teams that ship six excellent ones. Every feature kept in the MVP must be testable, demo-reliable, and citable to a specific empirical source. Features that do not meet all three criteria are out, regardless of how interesting they are.

PART III

Technical Architecture

3.1 System overview

Dyad is composed of six logical layers, executed across two physical surfaces (a macOS companion daemon for iMessage ingestion and an iOS mobile app for the user experience) plus a cloud-side orchestration tier built on GStack with persistent state in GBrain. The Hog runs as an enrichment service called by the orchestration tier. jo provides the broader personal-context layer, federated from the user's existing jo workspace.

The six layers, top-down:

- **L6 — Surface.** The iOS app. Three primary views: the Map (longitudinal visualization), the Atlas (emotion-tagged message stream), and the Brief (pre-conversation prep card).
- **L5 — Intervention.** Wise-intervention engine that produces reframes, repair drafts, and Brief content. Every output names its mechanism and cites its grounding literature.
- **L4 — Detection.** The six demo-critical detectors plus the ethical-refusal classifier. Each detector consumes state from L3 and produces typed output with calibrated confidence.
- **L3 — State estimation.** The three persistent models (self, partner-per-relationship, dyad-per-relationship), updated continuously as new signals arrive.
- **L2 — Signal extraction.** Pennebaker function-word parsing, LLM structured-feature extraction, action-identification classification, Higgins self-discrepancy tagging, NRC emotion lexicon pass.
- **L1 — Ingestion.** iMessage chat.db reader, future Discord/WhatsApp/Slack adapters, jo private-context federation, The Hog partner-external-context enrichment.

The orchestration through these layers is owned by GStack, which treats each layer as a service with a typed input and a typed output. This is the architectural decision that makes the build agent-parallelizable: each layer can be developed and tested independently against a fixture corpus, then composed.

3.2 Mobile-first iOS architecture

The mobile-first design choice creates one significant engineering constraint: iMessage data lives in chat.db on the user's Mac, in a sandboxed file the iOS app cannot read. This is solved with a two-surface architecture — a small macOS companion daemon does the ingestion, and the iOS app handles the entire user experience.

3.2.1 Two-surface architecture, in detail

The **macOS companion daemon** is a small Swift application with Full Disk Access permission. It reads chat.db, performs an initial historical sync at first run, then watches the file via FSEvents for incremental updates. Each new message is normalized into the Dyad message schema and pushed to the orchestration tier.

The **iOS app** is a SwiftUI native application. It does not ingest data itself; it consumes the state and detector outputs from the orchestration tier and renders the three core views. Authentication links the iOS device to the macOS daemon via a pairing flow at onboarding (QR code from Mac to phone, signed-token exchange).

For the twelve-hour hackathon build, the iOS app ships as a complete SwiftUI experience with mock-data fallback for testing, and the macOS daemon ships as a working but minimal CLI tool (reads chat.db, normalizes, posts to orchestration). The “daemon” is a Python script in disguise for the MVP; the Swift daemon is roadmap. This is the right trade: the demo audience sees the iOS app, not the Mac tool, so polish there matters more.

3.2.2 Why SwiftUI native, not React Native or PWA

Three reasons. First, the demo aesthetic matters: a native iOS app, shown on a real iPhone in a real iPhone frame, looks like a product. A React Native app with the wrong animations and off-spec native components looks like a hackathon project. Second, the system needs Local Authentication, push notifications, and Live Activities for the post-hackathon roadmap, all of which are far easier in SwiftUI. Third, the team is capable of shipping SwiftUI fast with agent assistance, and the surface area required for the demo is small (three screens plus a settings flow).

3.2.3 iOS app screen inventory

The MVP ships with these screens, in priority order:

- **Onboarding (3 cards).** Connect your Mac. Pick a relationship to begin with. Privacy explanation and consent.
- **Relationship list.** One row per dyad. Each row shows the emotional signature (last 7 days) as a small color band, the current rupture status (if any), and a numeric health indicator.
- **The Map.** The primary view. Scrollable longitudinal visualization, color-tagged by emotion, with rupture/repair events as discrete markers. Tap any point to drill into the underlying messages.
- **The Atlas.** Message-level view. Each message shows its primary emotion (color), intensity (saturation), and inferred secondary emotion (small label below). Long-press for citation.
- **The Brief.** Pre-conversation prep card. Generated on demand for a specific upcoming interaction. Includes unresolved threads, partner’s likely state, two opening drafts with predicted responses.
- **The Mirror (self-model view).** The uncomfortable feature. Surfaces what the system has learned about the user, including patterns the user is unlikely to acknowledge unprompted. Gated behind explicit opt-in.

3.3 End-to-end data flow

A single message’s journey through the system illustrates the architecture concretely. The flow below is for a partner-sent message arriving on the user’s phone:

- **Step 1 (instant, on Mac).** The macOS daemon’s FSEvents watcher fires. The daemon reads the new row from chat.db, normalizes timestamps, attaches conversation context (the last 20 messages in the thread), and POSTs the payload to the orchestration tier.
- **Step 2 (sub-second, orchestration).** GStack receives the payload, validates the schema, and routes to the L2 signal extraction pipeline. The function-word parser, NRC lexicon pass, and action-identification classifier run in parallel as deterministic compute — under 100ms total.
- **Step 3 (1-3 seconds, LLM).** An LLM extraction pass runs against the message and its 20-message context window, producing structured tags: emotion (primary + secondary + intensity + confidence), bid-or-response classification, horseman markers if any, validation markers, latency anomaly flag (compared to the dyad’s rolling baseline).

- **Step 4 (sub-second, state update).** The L3 state estimators update the three persistent models in GBrain. New facts are appended with citations to the originating message; confidence scores on prior facts are recomputed; the relationship-model's rolling metrics (PPR, bid response rate, 5:1 ratio, mirroring index) recompute over their respective windows.
- **Step 5 (asynchronous, enrichment).** The Hog is queried for any new external signals about the partner since the last enrichment cycle. New context is folded into the partner-model. This step is cached aggressively — The Hog is not queried per message, but per relationship per few hours.
- **Step 6 (event-driven, detection).** If the state update crosses a detection threshold (rupture firing, repair-window opening, phantom-third-party pattern matching), the L4 detector emits an event. The event is pushed to the iOS app via APNs as a Live Activity update or, for severe events, a Brief generation request.
- **Step 7 (on demand, surface).** When the user opens the iOS app, the Map and Atlas views consume the latest state from GBrain. Renders happen client-side from cached state — no LLM calls on view open. The Brief is generated lazily when the user requests one.

3.4 The signal extraction pipeline (L2 deep dive)

The signal extraction layer is the foundation of the entire product's epistemic credibility. Most consumer AI products that claim psychological insight produce outputs from a single LLM call over the raw text, which is fast, plausible, and unverifiable. Dyad does not do this. The L2 pipeline is deliberately heterogeneous — multiple parallel analyses, each grounded in a specific empirical method, combined into a single typed feature vector per message. Some are deterministic linguistic compute. Some are lexicon lookups. Some are LLM. Each carries its own confidence.

3.4.1 The function-word layer (Pennebaker)

James Pennebaker and collaborators spent thirty years showing that closed-class function words — pronouns, articles, prepositions, auxiliary verbs — are stronger psychological signal than content words. Pronoun shifts predict depression, deception, status dynamics, and relational distance. “I” usage spikes under threat; “we” correlates with relational health; second-person “you” in conflict is almost always blame.

The function-word parser is a fast deterministic pass over each message that produces these features:

- First-person singular pronoun rate (“I”, “me”, “my”, “mine”).
- First-person plural rate (“we”, “us”, “our”) and ratio against singular.
- Second-person rate (“you”, “your”), particularly elevated in blame contexts.
- Absolutist language markers (“always”, “never”, “everything”, “nothing”) — associated with depression and conflict escalation.
- Tentative language (“maybe”, “perhaps”, “sort of”) versus assertive language.
- Cognitive process words (“think”, “know”, “because”) indicating active reasoning.
- Affect words (positive and negative valence) from the NRC and AFINN lexicons.

These signals run in roughly 20-30 microseconds per message and do not require an LLM call, which makes the entire corpus tractable to analyze at ingest time.

3.4.2 The LLM structured-extraction layer

On top of the deterministic function-word layer, an LLM pass extracts features the closed-class layer cannot see. This pass is prompted with the message and a 20-message context window and returns structured JSON. The schema is:

```
{
  "bid_classification": {"is_bid": bool, "bid_type": str|null, "confidence": float},
  "response_classification": {"is_response_to_bid": bool, "quality":
"engaged|perfunctory|missed|hostile", "confidence": float},
  "horseman_markers": ["criticism", "contempt", "defensiveness", "stonewalling"],
  "validation_markers": {"acknowledges": bool, "paraphrases": bool, "asks_to_understand": bool},
  "primary_emotion": {"label": str, "intensity": "low|med|high", "confidence": float},
  "secondary_emotion_inference": {"surface": str, "underneath": str|null, "confidence": float},
  "action_identification_level": "low|high",
  "higgins_classification": "dejection|agitation|neutral|null",
  "topic_tags": [str],
  "latency_relative_to_baseline": {"z_score": float, "flag": "slow|fast|normal"},
  "clinical_flag": {"category": str|null, "confidence": float}
}
```

The LLM is prompted with a system message containing the relevant empirical definitions (Gottman's bid taxonomy, the four horsemen as Gottman defined them, the Plutchik wheel for emotions, Johnson's primary/secondary distinction for the secondary-emotion inference). Every claim in the output carries an explicit confidence score, and the system never displays a low-confidence claim to the user.

3.5 State objects in GBrain (L3 deep dive)

GBrain persists three logical objects per (user, relationship) tuple. Their schemas are described below in summary form; the engineering team should treat these as authoritative for schema design in the build.

3.5.1 The Self-Model

The Self-Model represents Dyad's longitudinal model of the user themselves, derived from how they behave across all their modeled relationships. Schema highlights:

- **Attachment indicators.** Behavioral signatures consistent with anxious, avoidant, secure, or disorganized patterns under threat, with confidence per dimension. Not displayed as a label to the user; used internally to inform other detectors.
- **Four-horsemen profile.** Rates at which the user produces each of the four horsemen markers in conflict contexts.
- **Bid responsiveness baseline.** Aggregate rate at which the user responds to bids across all modeled relationships, with per-relationship variance.
- **Action identification asymmetry.** The Vallacher-Wegner signature — degree to which the user describes their own behavior at low (mechanical) level versus others' behavior at high (dispositional) level.
- **Recurring relational templates.** Patterns that repeat across multiple relationships (used by the Phantom Third Party detector).

3.5.2 The Partner-Model (per relationship)

The Partner-Model is built from the partner's outbound messages within the dyad, augmented by The Hog's external context on the partner. It is explicitly probabilistic. Schema highlights:

- **Communication-style fingerprint.** Average message length, typical response latency distribution, vocabulary register, characteristic syntactic patterns.
- **Attachment-pattern inference.** Same dimensions as the Self-Model, with explicit lower confidence since the data is filtered through one specific relationship's dynamics.
- **External-context bundle.** Recent public activity from The Hog — what they've shared, engaged with, posted — with timestamps for correlation against in-relationship behavior changes.
- **Trigger profile.** Topics, phrasings, or contexts that produce above-baseline emotional reaction.
- **Bid signature.** The shape of how this person bids for connection — some bid through questions, some through shares, some through complaints. Critical for the bid-detection layer.

3.5.3 The Relationship-Model (the dyad itself)

The third object — and the one most products do not have — is the model of the relationship itself, as distinct from either party. Schema highlights:

- **PPR (perceived partner responsiveness), bidirectional.** Reis-Shaver-inspired aggregate metric, computed over rolling 30-day window.
- **5:1 positive/negative interaction ratio.** Gottman's magic ratio, computed over conflict-context messages.
- **Bid response rate, bidirectional.** The Feature 3 metric.
- **Asymmetric repair-labor index.** Who is doing the relational labor of initiating repairs.
- **Mirroring index.** Linguistic synchrony — the degree to which the two parties match each other's syntactic and rhythmic patterns. Decay in this index is one of the earliest detectable signals of relational drift.
- **Open loops.** Unresolved threads — conversations that ended without closure, ruptures that were never repaired, questions never answered.
- **Rupture/repair ledger.** Time-stamped record of every detected rupture and every detected repair, with current status (open / closed / window-expired).

3.6 Detection layer (L4) algorithm sketches

Each detector consumes the typed feature vectors from L2 plus the state from L3 and emits typed output with confidence. Algorithm sketches below are intended to be sufficient for the engineering team to implement; they are not pseudo-code, but they specify inputs, outputs, and the logic gate that produces each detector's claim.

3.6.1 Bid Response Asymmetry

Iterate over the dyad's message history. For each message classified as a bid by L2, look forward by N messages (N=5 in MVP) for a response from the other party. Classify the response as engaged / perfunctory / missed / hostile. Compute the engaged-response rate per direction. Return the two rates plus the gap and a confidence based on sample size.

3.6.2 Primary/Secondary Emotion Separation

Filter to messages tagged by L2 as conflict-context with surface emotion in the anger / contempt / disgust family. For each, examine the surrounding context (5 messages before, 5 after) and run a secondary LLM pass with a Johnson-EFT-grounded prompt asking what vulnerable emotion (hurt / fear / shame / loneliness) is being protected against. Surface only when confidence exceeds 0.7. Always cite the specific source messages.

3.6.3 Predictive Divergence

Take the user's draft message. Run two parallel LLM passes: one prompted with "summarize what this message is intended to communicate, given the user's recent messages as context," and one prompted with "summarize what the partner is likely to perceive this message as saying, given the partner-model's current state." Embed both summaries. Compute cosine distance. Render the distance visually, with the specific divergent phrases highlighted via attention-mapping or differential summarization.

3.6.4 Phantom Third Party

Detect a disproportionate-reaction event: emotional intensity of the user's response exceeds the rolling baseline for messages of this trigger-class by more than 1.5 standard deviations. Compute a behavioral fingerprint of the reaction (L2 features). Search the user's historical corpus across all modeled relationships for messages with similar fingerprints. If a cluster exists in a different relationship (different partner), and that cluster predates the current relationship, surface as candidate phantom-third-party. Require confidence above 0.8 to display. Never display without explicit user consent to surface this class of insight.

3.6.5 Ethical refusal

Run a continuous safety classifier (using public computational psychiatry models adapted to message context) for three categories: intimate-partner abuse markers, suicidality indicators, severe depression markers. When any classifier fires above its calibrated threshold, the L4 layer emits a refusal event that propagates up through L5 and L6, suppressing further analytical output and surfacing referral resources in the user-facing surface. The refusal is logged for the user's own records but never used as training signal.

3.7 Block integration

3.7.1 GStack as the orchestration spine

GStack runs the L2 → L3 → L4 pipeline on every ingested message, manages parallel detector execution, and handles event routing. Its role-based agent decomposition (planner, engineer, QA, release manager) maps cleanly to the team's parallel work streams during the build — the planner agent decomposes incoming feature requests into subtasks, the engineer agents implement against the typed interfaces, the QA agent runs the test corpus, the release manager handles deployment. This is exactly the use case GStack is built for.

3.7.2 GBrain as the longitudinal substrate

GBrain holds the three state objects per (user, relationship) tuple, the rupture/repair ledger, and the entity-resolution graph that links the same partner across multiple channels (iMessage Sarah and WhatsApp Sarah are the same Sarah). The entity resolution use case is non-trivial and showcases GBrain's strengths beyond simple chat memory — it is a real graph problem with timestamp-aware updates.

3.7.3 The Hog as partner-external-context provider

The Hog is queried per (relationship, partner) on a periodic schedule (default: every 6 hours) and on-demand when the user requests fresh context. The query pulls the partner's recent public activity, sentiment trends, named-entity engagement — what the partner has been thinking about, outside this relationship. This data is

folded into the partner-model with explicit provenance, so the user can see “your partner has been engaging with content about X for the past two weeks, and the change in their tone in your conversations correlates with that engagement.”

3.7.4 jo as the broader personal-context layer

jo provides the user’s personal context beyond just messages with the modeled partner — calendar events, photos that document shared experiences with the partner, personal notes that mention the partner, emails that cross-reference. This context enriches the Brief feature significantly: a Brief generated before a hard conversation with the user’s mother is dramatically more useful if jo can pull the photo album from last Thanksgiving and the calendar event for the upcoming visit. jo also handles the timing-intelligence layer — identifying when the user is likely in a state to receive a Brief versus when they are in the middle of something else.

PART IV

Ethical Architecture

Dyad operates on private communication between named real humans. This is a stronger ethical surface than most AI products encounter, and treating it as a design constraint rather than a marketing layer is what separates a fundable product from one that collapses on contact with a serious investor question. Ethics in this product is structural — built into the architecture — not policy bolted on top.

4.1 Five structural principles

4.1.1 Asymmetric intervention

Dyad only ever talks to the user. It never sends messages on the user's behalf, never auto-drafts and auto-sends, never addresses the other party directly, never reveals itself to the non-consenting partner. Every output is consumed by the user, for the user, with the user remaining the agent in every interaction. This is enforced architecturally: the system has no outbound-message capability at all, and the iOS app's "Repair Draft" feature produces text in a copy-buffer, not a send-button.

4.1.2 Your-side-only data access

Dyad observes only what the user already has access to: their own message history, their own calendar, their own photos. The partner's public footprint pulled through The Hog is, by definition, public. The system never accesses the partner's private accounts, never logs into the partner's services, never crosses the boundary between "information available to the user" and "information about the partner the user could not legally obtain."

4.1.3 The user is the primary diagnostic target

The Self-Model and the Mirror view are the product's deepest features by design. Most relationship-coaching tools focus on understanding the other person; Dyad focuses on what the user is doing that they cannot see. This is ethically load-bearing — it makes the product un-weaponizable, because the tool indicts the user as much as it explains the partner. A user who wants to use Dyad to "win" an argument with their partner will find the product turning the lens back on them. This is the feature, not a bug.

4.1.4 Clinical refusal as a first-class output

The ethical-refusal layer is shipped, not gestured at. When the system encounters patterns consistent with intimate-partner abuse, suicidality, or severe depression, it stops performing analytical work and surfaces appropriate referral resources. This is implemented as a real classifier built on the computational-psychiatry literature, not as a keyword filter. Drawing this line visibly on stage is a demo feature; drawing it consistently in production is the product's ethical floor.

4.1.5 Calibrated humility on humans

Every claim Dyad makes about a partner carries explicit confidence. The system is built to say "I don't know." The Phantom Third Party detector, which is structurally the most invasive feature, requires above-0.8 confidence to display, and the system actively suppresses low-confidence inferences in this category. This is the difference between a product that is psychologically serious and a product that produces confident horoscopes.

4.2 Data minimization and storage

Raw message data is processed and discarded; only the extracted feature vectors and aggregate state objects are persisted in GBrain. The original messages remain on the user's devices (Mac for iMessage, phone for the rendered experience), with the cloud-side state holding only the structured features required for the L3/L4/L5 layers. This is both a privacy posture and a practical one: storing every message in the cloud is unnecessary and a liability.

Model training: Dyad does not train on user data without explicit opt-in, and never trains on data flagged by the clinical-refusal classifier. The default state is no-training, with opt-in available for users who want to contribute to model improvement and understand what that entails.

4.3 Consent and transparency

The non-consenting partner is the central ethical concern in this product class. Dyad's position: the user has standing to reflect on their own communication, including communication with people who did not consent to that reflection being AI-mediated. This is the same standing a user has to keep a journal about their relationships, to discuss them with a therapist, or to re-read old messages. The position is defensible but not absolute, and the product is built to make the boundary visible:

- Insights about the partner are framed as inferences, not facts.
- Confidence is always visible.
- The system refuses to produce content that the user could weaponize against the partner — no “here is what to say to manipulate them” outputs, ever.
- The future “mutual mode” (post-MVP) supports explicit two-party opt-in where both members of the dyad use the product together; this is the consumer-positive expansion path.

PART V

Implementation Plan

5.1 Stack decisions, defended

Every technology choice below is the product of an explicit trade-off analysis. The defenses are written so that anyone on the team can answer “why this and not X” without calling the team lead.

iOS app	SwiftUI native. Reasons: aesthetic matters for demo, native animations and haptics are unmatched, team has capability with agent assistance, post-hackathon roadmap needs Live Activities and push that are far easier native.
macOS daemon	Python (MVP) / Swift (production). For twelve-hour build, Python script with sqlite3 against chat.db is sufficient and faster to ship. Swift daemon is post-hackathon work for the actual product.
Orchestration	GStack, treating each L-layer as a typed service. The planner/engineer/QA/release decomposition maps cleanly to parallel work streams during the build.
Persistent state	GBrain. Three state objects per (user, relationship), entity-resolution graph, rupture/repair ledger. Native to the stack.
External enrichment	The Hog, queried per relationship every 6 hours plus on-demand. Cached aggressively.
Personal context	jo, federated from the user’s existing jo workspace. Read-only access to calendar, photos, notes, non-message communication.
Signal extraction (deterministic)	Python with NLTK / spaCy for tokenization, NRC Emotion Lexicon and AFINN for affect, custom Pennebaker-equivalent dictionary for function words. Open-source baselines exist for all of these.
Signal extraction (LLM)	Claude or equivalent frontier model via the standard API. Structured output mode enforced via JSON schema. Context window: target message + 20-message surrounding context.
Visualization (mobile)	Swift Charts for the Map, custom SwiftUI Canvas for the Atlas color band, Lottie for transition animations. No web views in the demo path.

Demo data	Pre-loaded relationship corpus from team members with explicit consent, plus a Jobs/Sculley or Dorsey/Williams public-figure corpus as variance-reduction fallback.
Telemetry	Minimal during build. PostHog or equivalent for post-hackathon, with no message-content logging.

5.2 Team and agent distribution

The team operates as three parallel work streams, each owned by one human lead with one or more frontier coding agents executing specified tasks. The streams are defined to minimize cross-stream blocking: each stream produces artifacts that the other streams consume via typed interfaces, so streams move at their own pace and integrate at fixed checkpoints.

5.2.1 Stream A — Core Engine

Owns: the macOS ingestion daemon, the L2 signal extraction pipeline, the L3 state estimators, the L4 detectors. Outputs: a working backend that, given a chat.db file, produces a complete state and renders detector outputs for the iOS app to consume. Stream lead writes the prompts; agents implement the deterministic linguistic compute and the LLM-call scaffolding. This is the largest and most technically dense stream.

5.2.2 Stream B — Mobile / UX

Owns: the iOS SwiftUI app, the Map visualization, the Atlas rendering, the Brief generation flow, the Mirror screen. Outputs: a working iOS app that, given a state payload from Stream A, renders the six demo-critical features beautifully. Stream lead writes the screen specifications and animation choreography; agents implement the SwiftUI views against the typed state payload. The most demo-visible work.

5.2.3 Stream C — Demo & Pitch

Owns: the pre-loaded relationship corpus (team-member relationships with consent), the public-figure fallback corpus, the demo script with timings, the pitch deck and one-page handout, judge Q&A preparation, and the recovery protocols. This stream is owned by one person without agent assistance, because the work is judgment-heavy and benefits from a single consistent voice. The stream lead is also responsible for running practice demos at the integration checkpoints.

5.3 Hour-by-hour build schedule

The schedule is structured around three integration checkpoints (hours 4, 8, 11) where the three streams converge, exchange artifacts, and run an end-to-end test. The most important single hour is hour 11: it is reserved for cutting features rather than adding them.

Hour	Stream A — Core Engine	Stream B — Mobile/UX	Stream C — Demo & Pitch
------	------------------------	----------------------	-------------------------

0–1	Scaffolding. Repo, GStack init, GBrain schema scaffolds, iMessage chat.db reader prototype.	Scaffolding. SwiftUI project, navigation skeleton, three screens stubbed with mock data.	Locate consenting team-member relationships. Begin iMessage exports.
1–2	L1 ingestion working. Normalized message stream from chat.db. Function-word parser implemented.	Map visualization first pass with mock data. Animation choreography blocked out.	Build public-figure fallback corpus (Jobs/Sculley emails).
2–3	L2 LLM extraction pipeline. JSON schema enforced. Lexicon passes (NRC, AFINN).	Atlas message-level color tagging. Long-press citation. Empty-state design.	Demo script v1 drafted. Pitch outline drafted.
3–4	Checkpoint 1. Integration test: ingest sample corpus, run L2, dump features.	Checkpoint 1. iOS app loads mock state and renders Map + Atlas.	Checkpoint 1. First end-to-end dry run on test relationship.
4–5	L3 state estimators. Self/Partner/Relationship models updating from feature stream.	Brief screen implementation. Brief generation flow.	Practice demo timing. Identify weak narration moments.
5–6	L4 detectors 1-3 (Bid asymmetry, Primary/Secondary, Predictive Divergence). Confidence scoring.	Mirror screen. Self-Model surfacing UI.	One-page handout drafted (citations, architecture).
6–7	L4 detectors 4-5 (Phantom Third Party, Ethical Refusal). Threshold calibration.	Detector outputs wired into Map and Atlas. Live confidence indicators.	Pitch deck v1 complete. Q&A list drafted.
7–8	Checkpoint 2. Full pipeline on real relationship corpus. Detector calibration review.	Checkpoint 2. Full app on real state. Visual polish pass 1.	Checkpoint 2. Practice demo on real volunteer. Identify failures.
8–9	Tuning: which detectors fire reliably on which relationships? Cut the unreliable ones.	TTS narration overlay. Demo-mode UI distinct from working product.	Public-figure fallback demo path tested end to end.
9–10	Performance tuning. Eliminate latency in the demo path. Pre-warm caches.	Visual polish pass 2. Typography, color, motion. Demo-frame iPhone setup.	Run demo on three different relationships. Time each.
10–11	Backend freeze. No new features. Bug fixing only.	Frontend freeze. No new features. Bug fixing only.	Pitch rehearsal x3. Final adjustments to handout.

11–12	Checkpoint 3 & cut hour. Subtract any detector that does not fire reliably.	Checkpoint 3 & cut hour. Remove any UI element that distracts from the core five beats.	Checkpoint 3. Final demo run, final pitch run, handout printed. Team eats.
-------	---	---	---

5.4 The hour-11 ritual, in detail

Most teams in a twelve-hour build use the final hour to ship a last feature. Those teams lose. Dyad's hour 11 is structured as a cut, not an add, with a specific protocol:

- Run the full demo end-to-end on three different relationships from the pre-loaded corpus, with strict twelve-minute timing.
- For each detector, count how many of the three runs it fired on with a clean, defensible output. Detectors that fired on fewer than two of three are cut from the demo path, regardless of how much work went into them.
- For each narration beat, time it. Beats that take longer than three seconds to land are either rewritten shorter or cut.
- For each UI element on screen during a demo beat, ask: does removing this make the beat stronger? If yes, remove it.
- Re-run the demo end-to-end after the cuts. Time it. It should now be tighter than the pre-cut version, not just shorter.
- The final cut list is signed off by all three stream leads. No additions after sign-off.

5.5 Risk register

Risks are tracked explicitly with mitigations specified in advance. The risks below are ordered by expected impact times likelihood — the top three are the ones most likely to cost the hackathon if unmitigated.

Risk	Severity	Likelihood	Mitigation
Volunteer's data is too thin for live demo	High	Medium	Pre-loaded team-member corpus + public-figure corpus fallback. Branching demo script with three pre-tested entry points.
Phantom Third Party detector misfires on stage	High	Medium	Confidence threshold set high (0.8). Detector skipped silently if below threshold. Pre-tested on 5+ relationships before demo.

LLM rate limits or latency spike during demo	High	Low	All demo-path LLM calls pre-computed and cached. The Map and Atlas render from cache, not live calls. Predictive Divergence is the only live call on stage, with a 4-second timeout and a cached fallback.
Ethical refusal demo doesn't fire convincingly	Medium	Low	Pre-seeded synthetic conversation that reliably triggers the classifier. Demonstrated on a non-volunteer corpus to avoid false-positive on a real person.
Two streams diverge on typed interface	Medium	Medium	Three integration checkpoints (hours 4, 8, 11). Interface specifications frozen at hour 1 and changes require all-stream signoff.
iOS build fails on demo device at hour 11	High	Low	Two demo devices (phone A primary, phone B backup) kept warm-built from hour 8 onward. TestFlight install path also ready as third fallback.
Judge asks about training data / model sourcing	Low	High	Prepared answer naming the public datasets used (NRC Emotion Lexicon, AFINN, public Gottman annotation corpora) plus the team-member relationships used with consent. No undisclosed training data.
Judge asks about non-consenting partner	Medium	High	Prepared answer covering: user's standing to reflect on own communication, your-side-only access, no-message-sending architecture, mutual-mode roadmap.
Demo runs over time	Medium	Medium	Strict 7-minute hard limit on the demo portion. Practice until hitting 6:30 reliably. Cut content if needed.
Other team builds similar product in same room	Low	Very Low	Differentiate on (a) depth of literature backing, (b) ethical-refusal feature, (c) Map visualization, (d) Phantom Third Party. No team will hit all four.

PART VI

Demo Choreography

The demo is the single most important artifact the team produces. Everything else — the code, the architecture, the pitch — serves the seven-minute window in which a judge watches Dyad operate on a real relationship and decides whether this team is worth a YC conversation. This part of the document specifies the demo second-by-second, the variance-reduction strategy, the fallback paths, and the recovery protocols.

6.1 The seven-minute structure

The demo is exactly seven minutes long. Not because that is what the rules require, but because seven minutes is the longest a judge will give sustained attention to a single demo in a competitive room. The structure is paced to land four distinct “moment” beats — visual, factual, emotional, and structural — each within the attention budget of a sophisticated audience that has watched twenty demos already.

Time	On Screen	Narration / Action
0:00–0:15	iPhone in hand. Volunteer or judge data connects. Map begins to render and animate through time.	Silence. No narration. The visual lands first.
0:15–0:30	The Map completes its render. A number appears in the corner: “Bid Response: 71% / 44%.”	“What you’re seeing is fourteen months of this relationship rendered as emotional topography.”
0:30–1:00	Map highlights a rupture point 19 days ago. The repair-window indicator shows expired.	“The rupture three weeks ago at this point was never repaired. We can see it in every message since. The two of you have been operating in protected mode for nineteen days, and neither of you has named it.”
1:00–2:00	Bid asymmetry card scales up. Empirical thresholds (86% / 33%) displayed alongside.	“Forty years of relationship research from John Gottman established that the strongest predictor of relational outcomes is how often each person notices the other’s small attempts to connect. Stable couples respond to 86%. Failing relationships drop to 33%. You’re at 71%. They’re at 44%. You probably don’t know this. The gap has been quietly destroying the relationship, one missed bid at a time.”
2:00–3:00	Atlas view of a specific recent conflict. Surface emotion: anger. Underneath: fear of not mattering. Specific markers highlighted.	“On the surface, this fight was about the calendar. The linguistic signature on these eleven messages says it wasn’t. What they were actually upset about was not feeling prioritized. You can see it in these specific phrases. You read this as them being controlling. The model says they were asking to be chosen.”

3:00–4:00	Predictive Divergence view. Volunteer types a draft message. Two summaries appear side by side with the gap visualized.	“Before you send your next message, here’s what they will hear. On the left, what you intend to say. On the right, what the partner-model predicts they’ll perceive. The distance between those two summaries is the misunderstanding, rendered.”
4:00–4:45	Phantom Third Party card surfaces (if confidence permits). Pattern match against earlier-relationship corpus.	“One more thing. The reaction you had on Tuesday was disproportionate to the trigger. The pattern matches three earlier ruptures in this relationship, and looking back across your messages with your father from six months ago, it matches there too. You’re not fighting them right now. You’re fighting him through them.”
4:45–5:15	Switch to a pre-seeded conversation with concerning markers. System refuses to analyze. Referral resources appear.	“When the patterns we detect cross into clinical territory — abuse markers, suicidality, severe depression — the system declines to operate. We’re not therapy. We’re built to know when to step back. Few products in this room will do this on stage. We do it in production.”
5:15–6:30	Return to phone in hand. Switch to pitch mode. One-page handout visible.	The pitch section (see Part VII). Cover problem, why now, vertical expansion path, moat, team.
6:30–7:00	The closing line on the phone screen. Then a single sentence from the team lead.	“We built relational intelligence — the first software that operates on the real machinery of the relationships you can’t afford to lose. Working code, shipping today. We’re Dyad.”

6.2 Variance-reduction strategy

The single biggest threat to this demo is that the data the volunteer connects does not produce rich outputs. Counter this with three structural layers of fallback, each pre-tested:

6.2.1 Layer 1 — Pre-loaded team relationships

Five to eight real relationships from team members and consenting friends, ingested in advance and pre-validated. The team knows which detectors fire reliably on which relationship. If a volunteer’s data is thin, the demo pivots to “let’s use one we’ve already ingested so you can see the depth” without the awkward feeling of a fallback being deployed.

6.2.2 Layer 2 — Public-figure fallback corpus

Two pre-built corpora from public-figure relationships where the judge can verify claims against published material:

- **Primary: Steve Jobs & John Sculley.** A well-documented co-founder-style relationship with extensive published emails, interviews, and biographical material. The arc from collaboration to dismissal is visible

in surviving correspondence and recorded interviews. This corpus produces rich detector output across all six features, including a strong Phantom Third Party signal (Sculley's patterns from his Pepsi years recurring in his Apple conflicts).

- **Backup: Jack Dorsey & Ev Williams.** Another well-documented co-founder breakup with public correspondence, tweets, and post-hoc interviews. Useful as a second public-figure example if Jobs/Sculley feels overused.

6.2.3 Layer 3 — Guaranteed-firing detectors

Regardless of what data comes in, three detectors fire on essentially any meaningful corpus: bid response asymmetry, the four-horsemen presence/absence indicator, and the rupture-window status indicator. The demo script is structured so that the first ninety seconds rely only on these three detectors, with the higher-variance detectors (Primary/Secondary Separation, Predictive Divergence, Phantom Third Party) coming later, after the demo has already established credibility on the reliable beats.

6.3 Recovery protocols

If something fails on stage, the team has rehearsed responses. The principle is: never apologize, never freeze, never lose narrative momentum. The demo continues, and the failure becomes a feature of the team's composure rather than a feature of the product's instability.

6.3.1 If a detector misfires

The narration includes confidence scores. If a detector produces a low-confidence output, the presenter says: "Here the model has lower confidence — you can see it's flagged 0.61 — so it won't commit to the claim. The Phantom Third Party detector is designed to be loud only when it's certain." This converts a failure into a demonstration of calibrated humility, which is itself a differentiator.

6.3.2 If the iOS app crashes

Two demo devices are kept warm-built from hour 8. The primary device is in the presenter's hand; the backup is in the co-presenter's pocket. If the primary crashes mid-demo, the co-presenter hands the backup device forward without breaking eye contact with the judge. The presenter continues narration through the swap as if rehearsed (because it is).

6.3.3 If the LLM call times out

The Predictive Divergence feature is the only live LLM call on stage. It has a 4-second timeout. If the timeout fires, the cached fallback output renders instead, with the presenter saying: "The live model is rendering — here's what it produced on the prior message in this thread for comparison."

6.3.4 If a judge asks a hostile question mid-demo

Stop the demo. Engage the question fully. Resume the demo from the prior beat, not from where the interruption happened. Most judges respect this; the ones who don't are not the judges to optimize for. The presenter is trained to identify the hostile-question class and respond from the prepared Q&A list (Part VII, section 7.4).

6.4 Pre-demo checklist

Run this checklist five minutes before stepping on stage. Anyone on the team can call it. The team lead signs off:

- Both demo iPhones unlocked, on full charge, on the right Wi-Fi.
- Both iPhones display the Dyad app, navigated to onboarding (not the home screen).
- Pre-loaded corpus is hot in the cache — ran a test query in the last 5 minutes.
- Public-figure fallback corpus is hot in the cache.
- The Hog query for the partner has been pre-warmed within the last 30 minutes.
- Predictive Divergence has been tested on stage with the actual lighting and audio in the last 15 minutes.
- The ethical-refusal pre-seeded conversation is one tap away from the current state.
- One-page handout is printed and on the table where the judges can see it.
- Co-presenter is positioned to hand off the backup phone if needed.
- Pitch script is in the presenter's working memory, not on a card.

PART VII

The Pitch

The pitch occupies the final ninety seconds of the demo window plus the Q&A period that follows. It is structured as a tight verbal walk through five beats — problem, why-now, vertical-expansion path, moat, team — with prepared responses for the most likely judge questions.

7.1 The opening, memorized verbatim

“Most adult suffering happens in dyads. Estranged parents. Co-founder breakups. Marriages that ended without warning. Friendships that ghosted. All of it shares one structural feature: the dynamics that govern the relationship are invisible to the people inside it, and by the time someone realizes what was happening, the window has closed.”

This opening is non-negotiable. It is the sentence the judge will remember if they remember nothing else from the demo. It is delivered slowly, with eye contact, before any technology is shown. Every other team in the room will open by describing their technology. Dyad opens by describing a feeling the judge has had.

7.2 The why-now beat

Three things became simultaneously true in the last twelve months: foundation models can perform linguistic analysis on intimate communication with calibrated confidence; persistent memory infrastructure has matured enough that longitudinal modeling is cheap; and the consumer permission structure for personal AI has normalized giving an agent access to private channels. None of these were true two years ago. All three are required. This is the window.

7.3 The vertical-expansion path

Dyad's consumer wedge is one wedge in a larger market that the same engine extends into. The expansion path, delivered as a single sentence:

“Consumer relationships first. Then co-founder relationship monitoring — and you know what fraction of YC companies die because of co-founder conflict. Then high-stakes negotiation preparation. Then sales-call relational intelligence. Then mediation infrastructure. Then therapy augmentation. Same engine. Different surfaces. The moat is the longitudinal data on real dyads, which nobody else can replicate.”

The co-founder line is intentionally pointed. It is a specific, visceral pain point the judges have lived through — either personally or through portfolio companies. Naming it out loud changes the temperature of the room.

7.4 Q&A preparation

Twelve questions are pre-rehearsed. The team is trained to answer each in under twenty seconds, with a longer follow-up if the judge probes. The team has a single designated responder for each category — technical, business, ethical, scientific — to avoid the appearance of confusion when the team huddles.

7.4.1 Technical depth questions

- **“What’s actually new here, technically?”** The composition of three persistent state objects (self, partner-per-relationship, dyad) plus longitudinal entity-resolution across channels plus calibrated-confidence extraction grounded in Pennebaker function-word linguistics. Each piece exists somewhere; no shipped product composes them.
- **“Why not just use GPT to do it all?”** Because LLM-only psychological inference is unverifiable. The Pennebaker layer is deterministic and citable to forty years of replicated research. We use LLMs for the structured extraction layer on top, but the foundation is closed-class linguistic analysis that doesn’t get gamed.
- **“How accurate is the model on real data?”** Bid response asymmetry: within 5% of human-annotated ground truth on our test corpus. Primary/Secondary separation: 0.78 agreement with two independent EFT-trained annotators. Phantom Third Party: deliberately set with high confidence threshold; precision over recall.

7.4.2 Business / fundability questions

- **“What’s the business model?”** Consumer subscription at \$20/month, similar to therapy adjuncts and premium personal AI. Enterprise/co-founder tier at \$200/month per active dyad. Long-term: licensing the relationship engine into mediation and therapy workflows.
- **“Who actually pays for this?”** People in high-stakes relationships they cannot afford to lose. The consumer beachhead is people considering or in therapy — they pay \$200/session for less continuous insight. The co-founder tier is sold to companies whose founders are drifting; HR doesn’t want to know, but the founders do.
- **“Why won’t a big platform clone this?”** The moat is longitudinal data on dyads, which compounds with use. Three months of relational history is an asset no competitor can replicate at launch. Network effects are asymmetric — the more you use it on one relationship, the harder it is to switch.

7.4.3 Ethical questions

- **“What about the partner who didn’t consent?”** The user has standing to reflect on their own communication, including with people who didn’t consent to that reflection being AI-mediated — the same standing they have to journal, to discuss with a therapist, to re-read old messages. We never send messages, never impersonate, never access the partner’s private channels. Mutual mode for two-party opt-in is roadmap.
- **“Couldn’t this be used to manipulate?”** Structurally, no. Every output we produce indicts the user as much as it explains the partner — the self-model and the Mirror view are the deepest features. We refuse to produce “how to manipulate them” outputs at the prompt level. We are explicitly the wrong tool for a user who wants to win a fight.
- **“What if you’re wrong about a serious thing?”** Every claim carries confidence. The clinical-refusal layer stops the system entirely when serious patterns appear. We refer to real resources. We never present probabilistic inferences about humans with the confidence interface of search results.

7.4.4 Scientific grounding questions

- **“Which research is this actually based on?”** The full bibliography is in our handout. Core: Gottman on rupture and repair, Reis & Shaver on perceived partner responsiveness, Sue Johnson’s EFT on primary versus secondary emotion, Mikulincer & Shaver on attachment under threat, Walton & Wilson on wise interventions, Pennebaker on function-word linguistics, Wachtel on cyclical psychodynamics, Coppersmith and De Choudhury on computational psychiatry for the refusal layer.

- **“Aren’t some of these contested?”** Yes. Gottman’s 90% accuracy claim has been challenged on out-of-sample data; polyvagal theory’s mechanistic claims are debated; attachment theory has known operationalization issues. We treat all of these as informed priors with calibrated confidence, not ground truth. The function-word linguistics layer (Pennebaker) is the most empirically stable foundation and we lean on it hardest.

7.5 The closing line

“We built relational intelligence — the first software that operates on the real machinery of the relationships you can’t afford to lose. Working code, shipping today. We’re Dyad.”

This is the line the judges repeat to each other after we leave the room. It is therefore the most carefully engineered sentence in the entire pitch. It is delivered with the phone still in hand, the Map still on screen, and the team standing together.

PART VIII

Post-Hackathon

The hackathon is the entry point, not the destination. This section specifies what happens in the seventy-two hours after the demo, the first thirty days, and the ninety-day plan. The assumption is that the YC interview happens within two weeks of the demo, and the goal of the seventy-two-hour window is to be ready when the call comes in.

8.1 The 72 hours after the demo

- Convert the demo build into a working private beta. Real Swift daemon for the Mac, not the Python prototype.
- Open a private beta wait list. Cap at 100 users. Source first users from co-founder networks, therapists, and YC alumni groups specifically.
- Write the founding-story version of the pitch. The seven-minute version is for judges; the longer version is for the YC partner interview.
- Set up the data infrastructure for real users — encryption at rest, access logging, audit trail. The hackathon build is explicit prototype quality; the post-hackathon build must be operationally serious.
- Identify the one critical engineering hire: a clinical research lead who can validate the detection layer against expert annotation. This person makes the difference between “impressive demo” and “defensible product.”

8.2 The first 30 days

- Ship two of the deferred features: cross-relationship pattern transfer (the “you have this same fight with three different people” output), and the four-horsemen live classifier as a surfaced metric.
- Run a validation study against expert annotation on 100 message exchanges from consenting beta users. Publish the precision/recall numbers internally; share with investors on request.
- Build the consent flow for mutual mode. Two-party opt-in for couples, co-founders, family. This is the consumer-positive expansion that makes the ethical narrative obvious.
- Add Discord and WhatsApp ingestion adapters. iMessage primary, but the architecture supports multi-channel from day one.
- Hire the clinical research lead.

8.3 The 90-day plan

- Launch into limited public availability with 1,000 users. Pricing tier: \$20/month consumer.
- Begin the co-founder relationship monitoring pilot with 5 YC-portfolio companies, sourced through the post-hackathon network.
- Ship attachment-style classification as an opt-in user-facing feature with consent flow and clinical framing.
- Begin development of the therapy-augmentation surface — the version of Dyad where a licensed therapist is in the loop and the system supports their work between sessions.

- Series Seed close.

8.4 The eighteen-month picture

By the eighteen-month mark, Dyad is the relational-intelligence layer underneath three distinct surfaces: the consumer product, the co-founder-monitoring product, and the therapy-augmentation product. The longitudinal data on dyads — tens of thousands of modeled relationships, with consent, with calibrated detector outputs — is an asset no competitor can replicate. The research collaboration with at least two university labs has produced one peer-reviewed validation paper. The category “relational intelligence” is the team’s phrase, used by analysts, written about by journalists, and the obvious label for whatever comes next.

APPENDIX A

Research Bibliography

Every detector in the system is grounded in a specific literature. This bibliography is the reference list for the engineering team, the basis of the citations in the one-page handout, and the answer to the most common scientific-grounding question. The list is intentionally not exhaustive; it is the essential reading.

A.1 Foundational relationship science

- **Gottman, J. M., & Levenson, R. W. (2000).** “The timing of divorce: Predicting when a couple will divorce over a 14-year period.” *Journal of Marriage and Family*, 62(3). The longitudinal study that produced the predictive validity claims.
- **Gottman, J. M. (1999).** *The Seven Principles for Making Marriage Work*. The bid taxonomy and the five-to-one ratio originate here.
- **Reis, H. T., & Shaver, P. (1988).** “Intimacy as an interpersonal process.” In *Handbook of Personal Relationships*. The foundational PPR paper.
- **Reis, H. T., Clark, M. S., & Holmes, J. G. (2004).** “Perceived partner responsiveness as an organizing construct.” The empirical consolidation of PPR as the dominant relational health metric.
- **Tronick, E. (2007).** *The Neurobehavioral and Social-Emotional Development of Infants and Children*. The still-face paradigm and the rupture-repair framework.

A.2 Emotion-focused frameworks

- **Johnson, S. M. (2008).** *Hold Me Tight*. The accessible introduction to EFT and the primary/secondary emotion distinction.
- **Johnson, S. M. (2019).** *Attachment Theory in Practice*. The clinical operationalization of attachment and EFT.
- **Plutchik, R. (2001).** “The nature of emotions.” *American Scientist*, 89(4). The wheel of eight primary emotions that the Emotion Atlas uses.
- **Ekman, P. (1992).** “An argument for basic emotions.” *Cognition and Emotion*, 6(3). The intensity-gradient grounding.
- **Higgins, E. T. (1987).** “Self-discrepancy: A theory relating self and affect.” *Psychological Review*, 94(3). The dejection-versus-agitation family classification.

A.3 Attachment and dynamics

- **Mikulincer, M., & Shaver, P. R. (2016).** *Attachment in Adulthood* (2nd ed.). The reference text on adult attachment, with behavioral markers operationalizable in text.
- **Wachtel, P. L. (2014).** *Cyclical Psychodynamics*. The dyad-as-unit-of-pathology framework underneath the Phantom Third Party detector.
- **Murray, S. L., & Holmes, J. G. (2009).** “The architecture of interdependent minds.” *Personality and Social Psychology Review*, 13(1). Risk regulation theory.

A.4 Computational linguistics and psychiatry

- **Pennebaker, J. W., et al. (2003).** “Linguistic styles: Language use as an individual difference.” The function-word framework that grounds Dyad’s L2 deterministic layer.
- **Pennebaker, J. W. (2011).** *The Secret Life of Pronouns*. The accessible book-length treatment.
- **Mohammad, S. M., & Turney, P. D. (2013).** “Crowd-sourcing a word-emotion association lexicon.” The NRC Emotion Lexicon paper.
- **Coppersmith, G., et al. (2018).** “Natural language processing of social media as screening for suicide risk.” The methodological foundation of the clinical-refusal layer.
- **De Choudhury, M., et al. (2013).** “Predicting depression via social media.” The companion methodology for the depression-marker classifier.

A.5 Intervention design

- **Walton, G. M., & Wilson, T. D. (2018).** “Wise interventions: Psychological remedies for social and personal problems.” *Psychological Review*, 125(5). The framework for the intervention layer.
- **Aronson, E., et al. (1995).** “Self-affirmation: Reducing dissonance and other identity threats.” The ground for the reframe-design principle.

APPENDIX B

Linguistic Markers Reference

This appendix is the engineering team's reference for the linguistic markers extracted in L2. Each marker is named, defined, and tied to the detector that consumes it. Programmers building the L2 layer use this as a specification.

B.1 Function-word markers (Pennebaker family)

FW_I	Rate of first-person singular pronouns. Elevated rates correlate with depression, threat, status loss. Feeds: self-model, Higgins classifier.
FW_WE	Rate of first-person plural pronouns. Healthy relational indicator. Decline correlates with relational drift. Feeds: relationship-model, mirroring index.
FW_YOU	Rate of second-person pronouns. In conflict context, particularly elevated in blame patterns. Feeds: four-horsemen (criticism marker).
FW_ABS	Rate of absolutist language: always, never, everything, nothing. Marker of cognitive distortion and escalation. Feeds: rupture detector, four-horsemen.
FW_TENT	Rate of tentative language: maybe, perhaps, sort of. Marker of low certainty or non-commitment. Feeds: bid-quality classifier.
FW_COG	Rate of cognitive process words: think, know, because. Marker of active reasoning. Feeds: action identification level.

B.2 Affect markers (lexicon-based)

NRC_EMO_*	Eight binary indicators (one per Plutchik primary) plus two valence indicators (positive/negative) from NRC. Feeds: Emotion Atlas primary tagging.
AFINN_VAL	Continuous valence score from AFINN. Feeds: Map emotional-temperature line.
INTENSIFIER_RATE	Rate of intensifiers (very, really, totally, completely). Feeds: Emotion Atlas intensity dimension.

B.3 LLM-extracted markers

BID_CLASS	Whether the message is a bid for connection, and if so, what type (observation, question, share, request). Feeds: bid response asymmetry detector.
------------------	--

RESPONSE_CLASS	Quality of response to a preceding bid: engaged, perfunctory, missed, hostile. Feeds: bid response asymmetry detector.
HORSEMAN_*	Four binary markers for criticism, contempt, defensiveness, stonewalling. Feeds: Map rupture markers, four-horsemen metric.
VALIDATION_*	Three binary markers for acknowledgment, paraphrasing, asking-to-understand. Feeds: PPR detector.
PRIMARY_EMOTION_L LM	Primary emotion with intensity and confidence, refined from the lexicon-based first pass. Feeds: Emotion Atlas.
SECONDARY_EMOTIO N_INFERENCE	Underlying vulnerable emotion beneath surface emotion, when applicable. Feeds: Primary/Secondary separation detector.
ACTION_ID_LEVEL	Vallacher-Wegner level of the action description: low (mechanical) or high (dispositional). Feeds: self-model asymmetry.
HIGGINS_FAMILY	Dejection-family or agitation-family emotional substrate. Feeds: Primary/Secondary inference.
CLINICAL_FLAG	Category of clinical concern, if any: abuse, suicidality, severe-depression. Feeds: ethical-refusal layer.

APPENDIX C

Demo Recovery Protocols

This appendix is the script the team trains on for failure scenarios. Each protocol is one paragraph, memorized, and rehearsed during checkpoint 3.

C.1 Volunteer data is thin

“Let’s use one of the relationships we’ve already ingested for testing — it’ll show you the full depth. [Open pre-loaded team relationship.] This is one of our team members, with consent. Watch what the system surfaces.”

C.2 Detector misfires with low confidence

“Here the model has lower confidence — you can see it’s flagged 0.61. The system is designed to be loud only when it’s certain. That’s the difference between a psychologically serious product and a horoscope.”

C.3 iOS app crashes

“Let me switch to my colleague’s device while the primary recovers. [Co-presenter hands over backup phone.] Same state, same demo. The data lives in our backend, not on the phone.”

C.4 Live LLM call times out

“The live model is still rendering — here’s what it produced on the prior message for comparison. [Cached output appears.] In production this latency is under two seconds; on demo Wi-Fi we’re slower.”

C.5 Hostile judge question mid-demo

“That’s the right question to ask. Let me answer it fully. [Pause demo. Answer from the Q&A list. Take the follow-up if there is one.] Returning to the demo — we were at the bid asymmetry beat.”

C.6 The Phantom Third Party fires on the wrong person

“Interesting. The detector found a pattern match, but you’re right to push back — the confidence here is only 0.74, below our usual demo threshold. This is exactly why we show the confidence and let the user decide whether the claim is useful.”

C.7 The ethical-refusal demo doesn’t trigger

“Let me load the corpus we use for testing the refusal layer. [Switch to pre-seeded conversation.] This is a synthetic exchange designed to test the classifier. Watch the system refuse to operate.”

END OF DOCUMENT

Build it

This document is the specification, not the product. The product is the seven-minute experience in the judging room and the ninety-day company that follows. Everything in these pages exists to maximize the probability that the right team builds the right thing in the available time, demonstrates it to the right people, and is ready when the call comes in.

The hackathon ends when the demo ends. The work begins immediately after. Be ready.

— Dyad team